RL Ch.1 Intro

4/9/26

I. RL is a computational method that focuses on maximizing reward by a learner discovering what actions lead to the best outcome

A. The biggest focus is on trial-and-error search (finding good reward) AND delayed reward (reward may be great in a few actions)

B. In this learning paradigm, there is an exploration vs exploitation tradeoff.
1. The agent must explore options to discover the best path
2. However, it must exploit previous knowledge to maximize reward and not exhaustive search
3. A balance must form to complete the task

C. RL has agents that can influence the state of its environment to achieve its goals

II. The four main subelements in RL: policy, reward signal, value function, and environment model

A. Policy defines the agent's behavior at some time

B. Reward signal defines the goals of the problem (sent immediately each time step). Helps alter the policy as a primary signal.

C. Value function defines what is good in the long run rather than the immediate reward signal. A state can have high value but low reward because the states after have high reward. Secondary and reliant on reward, but are most consulted when making decisions. Often computed over agent lifetime & source of significant computation

D. Model of the envir. mimics the environment's behavior to infer how the envir. will behave. Given a state and action, the model will predict a next state/reward/value. RL models may not implement a model, hence no planning, and thus it is a trial-and-error learner. (model-free vs model-based)

III. RL learns from an environment/model

A. In contrast to evolution, who only considers total success or failure of policy, RL takes into account states

B. Page 8, see tic-tac-toe example w/ learning rate α

Scanned with CamScanner

Exercises:

1.1: In the case of self-play, the two models learning would likely learn slower/worse than if against a competent player. Both would make random moves, and some bad moved may be learned as good because the other model fails to catch on, unlike a sentient player. They would probably still converge, but slower.

Part I: Tabular Solution Methods

I. Multi-arm Bandits

A. An n-arm bandit problem consists of a single state (situation) with n arms (actions) with the goal to maximize reward from a static probability distr. of said actions (over some time period.

1. Values (reward) of states are assumed unk or perhaps w/ estimates
2. If given the time, instead of being greedy, may be better to explore for best greedy option long term
3. This is the <u>exploitation vs. exploration</u> problem in RL, of which there are many balancing methods

B. Exploitation vs Exploration in Multi-arm Bandit

action-value methods

1. Naive way to get value is taking the avg. reward over many steps t. We start the value at 0 $\hat{q}(a)$ and, as selection # $\to \infty$, we get the true $q(a)$
   a. There is greedy and ε-greedy (always get best vs. select random ε% of time)
   b. Most of the time, ε-greedy is better if given time because RL, by nature of the problems, must balance explor. vs exploit.
2. The previous learning revelation can be summarized as $V_{old} + \frac{1}{k}(V_{new} - V_{old})$, taking a similar learning view as before

general form

$\to$ NewEstimate = OldEstimate + StepSize [Target − OldEstimate]

(StepSize: learn rate α; Target − OldEstimate: error)

3. What happens in a non-stationary environment (values constantly change for reward)?

Scanned with CamScanner

a. Easiest way is to keep α constant so new learning gets more heavily weighted than aggregated old (unlike fading α).

b. Over time, rewards from older runs have less influence exponentially

(by expanding $Q_{k+1} = Q_k + \alpha[R_k - Q_k]$) $\rightarrow Q_{k+1} = (1-\alpha)^k Q_1 + \sum_{i=1}^{k} \alpha(1-\alpha)^{k-i} R_i$

c. α constant results in a first selection bias, aka $Q_1(a)$, aka the value that is dev selected before the action is tried.

i. Taking advantage of this, optimistic initial values set value high above the known value result, forcing greedy to explore each action at the start b/c vals. keep ↓.

ii. Not as useful for non-stationary, general problems

4. When exploring, if we have estimate values, we should explore actions with the most promise to be efficient

Upper Confidence Bound (UCB)

a. The best action to explore would be like

$$A_t = \underset{a}{\operatorname{argmax}} \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

c: confidence; $\ln t$: slowly build desire to test action; $N_t(a)$: # of times action tried

b. This equation takes into account we may already know a lot about an action, or maybe not.

c. Not used much in advanced RL b/c non-stationary problems + large state spaces

5. Can define a new "probability of taking an action" $\pi_t(a)$ Considering a preference value $H_t(a)$ as:

$$\frac{e^{H_t(a)}}{\sum_{b=1}^{n} e^{H_t(b)}} = \pi_t(a)$$

and we can update preference of every action each time step w/:

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)) \quad \text{and}$$

$$H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a) \quad \text{for } \forall a \neq A_t$$

where $\bar{R}_t$ is the average of all rewards including step t (not exclusive to an action)

This is all NON-Associative, where there is no state change.

Next, we consider Associative where a policy must be learned to consider state

Scanned with CamScanner